AFRILANG Stdlib

std/c/netip

Français. Bibliothèque AFRILANG ‘std/c/netip‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : parseOctets, toInt, fromInt, isPrivate, isValid, sameSubnet, broadcast.
‘import ”std/c/netip”‘ · ‘use netip‘
- ‘parseOctets(ip text) → list number‘ - ‘toInt(ip text) → number‘ - ‘fromInt(n number) → text‘ - ‘isPrivate(ip text) → bool‘ - ‘isValid(ip text) → bool‘ - ‘sameSubnet(a text, b text, mask number) → bool‘ - ‘broadcast(ip text, mask number) → text‘ **English.** AFRILANG library ‘std/c/netip‘ (tier complex, category complex). Exports 7 function(s): parseOctets, toInt, fromInt, isPrivate, isValid, sameSubnet, broadcast.
‘import ”std/c/netip”‘ · ‘use netip‘
- ‘parseOctets(ip text) → list number‘ - ‘toInt(ip text) → number‘ - ‘fromInt(n number) → text‘ - ‘isPrivate(ip text) → bool‘ - ‘isValid(ip text) → bool‘ - ‘sameSubnet(a text, b text, mask number) → bool‘ - ‘broadcast(ip text, mask number) → text‘

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/netip‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : parseOctets, toInt, fromInt, isPrivate, isValid, sameSubnet, broadcast.

‘import ”std/c/netip”‘ · ‘use netip‘

```
- `parseOctets(ip text) → list number`
- `toInt(ip text) → number`
- `fromInt(n number) → text`
- `isPrivate(ip text) → bool`
- `isValid(ip text) → bool`
- `sameSubnet(a text, b text, mask number) → bool`
- `broadcast(ip text, mask number) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/netip"
use netip
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés — graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 parseOctets(ip text) → list•
 toInt(ip text) → number•
 fromInt(n number) → text•
 isPrivate(ip text) → bool•
 isValid(ip text) → bool•
 sameSubnet(a text, b text, mask number) → bool•
 broadcast(ip text, mask number) → text
```

4. Exemples d’utilisation

```
import "std/c/netip"
use netip
```

```
create result = parseOctets(/* args */)
say result
```

```
create result = toInt(/* args */)
say result
```

```
create result = fromInt(/* args */)
say result
```

```
create result = isPrivate(/* args */)
say result
```

```
create result = isValid(/* args */)
say result
```

```
create result = sameSubnet(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/netip"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module netip

```
function parseOctets(ip text) returns list number
end
```

```
function toInt(ip text) returns number
end
```

```
function fromInt(n number) returns text
end
```

```
function isPrivate(ip text) returns bool
end
```

```
function isValid(ip text) returns bool
end
```

```
function sameSubnet(a text, b text, mask number) returns bool
end
```

```
function broadcast(ip text, mask number) returns text
end
end
```

English

1. Overview

AFRILANG library ‘std/c/netip‘ (tier complex, category complex). Exports 7 function(s): parseOctets, toInt, fromInt, isPrivate, isValid, sameSubnet, broadcast.

‘import ”std/c/netip”‘ · ‘use netip‘

```
- `parseOctets(ip text) → list number`
- `toInt(ip text) → number`
- `fromInt(n number) → text`
- `isPrivate(ip text) → bool`
- `isValid(ip text) → bool`
- `sameSubnet(a text, b text, mask number) → bool`
- `broadcast(ip text, mask number) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/netip"
use netip
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 parseOctets(ip text) → list•
 toInt(ip text) → number•
 fromInt(n number) → text•
 isPrivate(ip text) → bool•
 isValid(ip text) → bool•
 sameSubnet(a text, b text, mask number) → bool•
 broadcast(ip text, mask number) → text
```

4. Usage examples

```
import "std/c/netip"
use netip
```

```
create result = parseOctets(/* args */)
say result
```

```
create result = toInt(/* args */)
say result
```

```
create result = fromInt(/* args */)
say result
```

```
create result = isPrivate(/* args */)
say result
```

```
create result = isValid(/* args */)
say result
```

```
create result = sameSubnet(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/netip"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module netip

```
function parseOctets(ip text) returns list number
end
```

```
function toInt(ip text) returns number
end
```

```
function fromInt(n number) returns text
end
```

```
function isPrivate(ip text) returns bool
end
```

```
function isValid(ip text) returns bool
end
```

```
function sameSubnet(a text, b text, mask number) returns bool
end
```

```
function broadcast(ip text, mask number) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/netip"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/netip/

Source (extrait)

```
module netip

function parseOctets(ip text) returns list number
end

function toInt(ip text) returns number
```

```
end

function fromInt(n number) returns text
end

function isPrivate(ip text) returns bool
end

function isValid(ip text) returns bool
end

function sameSubnet(a text, b text, mask number) returns bool
end

function broadcast(ip text, mask number) returns text
end
end
```